1. The Heavy Hitters (For Logic & Math)

GPT 5.2 (OpenAI)

- **Best for: Pure Mathematics & Spreadsheets.**
- **Why:** It holds the perfect 100% score on the AIME 2025 (American Invitational Math Exam). If you need an absolute row count or financial modeling without a single error, GPT 5.2 in **"Pro" or "Thinking" mode** is the gold standard.

- **The Catch:** It has a smaller context window (400k) compared to Gemini, so it might struggle with massive multi-gigabyte files.

Gemini 3.1 Pro (Google)

- **Best for: Massive Context & "No Truncation" Research.**

- **Why:** It fixed the infamous "output truncation" bug. It's the highest scorer on abstract reasoning (ARC-AGI-2).

- **The Catch:** It's slower than Flash. Use this when the CSV is huge and you need the AI to "hold" the entire 1M+ token dataset in active memory without losing focus on the bottom of the file.

Claude Opus 4.6

- **Best for: Complex Coding & Multi-Agent Workflows.**
- **Why:** It's the "Engineer's Model." It has the highest output ceiling (128k tokens), meaning it can write an entire corrected version of your file back to you without stopping.
- **The Catch:** Most expensive and sometimes "over-thinks" simple counting tasks.

---

2. The Speedsters (For Volume & Vision)

Gemini 3 Flash

- **Best for: General Workhorse Tasks & Speed.**
- **Why:** Surprisingly, it actually beats the Pro version in terminal-based coding benchmarks this year. It's the "budget" choice that doesn't feel budget.

- **The Catch:** It uses "Modular Thinking," so if you don't set it to "High" or "Max" reasoning, it might take a shortcut and guess your row count (giving you that rounded 1000).

Gemini Flash (Text & Vision)

- **Best for: Visual Data & Charts.**
- **Why:** It uses **Agentic Vision**. If you upload a *screenshot* of a spreadsheet or a complex PDF chart, it doesn't just "look" at it—it uses Python to zoom in, crop, and verify pixel-level details.

---

3. Recommended "Verification Mode" Setup

To fix your specific counting error (1000 vs 1900), here is how to use these models effectively:

Step 1: The "Lead" (GPT 5.2 Pro)

Ask **GPT 5.2** to generate the counting logic.

> *"Use your Extended Thinking mode to write a Python script that counts the rows in this CSV. Output the total as a verified integer."*

Step 2: The "Auditor" (Gemini 3.1 Pro)

Feed the file to **Gemini 3.1 Pro** and say:

> *"I have been told there are [Result from Step 1] rows. Use your 1M context window to locate the specific data in the last 10 rows. If the last row index does not match the count, flag the discrepancy."*

Step 3: The "Visual Check" (Gemini 3 Flash Vision)

---

1. The "Code-First" Solution (High Accuracy)

The only way to get **100% accuracy** with CSVs is to stop the LLM from "reading" the rows and force it to "calculate" them. If your platform has a **Code Interpreter** or **Advanced Data Analysis** mode, use it.

The Prompt Strategy: Instead of asking "How many rows are in this file?", use a prompt that

forces a deterministic tool:

> "Run a Python script using Pandas to load this CSV and output `df.shape`. Do not estimate; provide the exact integer count from the code output."

---

2. Implement a Verification & Feedback Loop

To ensure accuracy, you can build a **"Chain of Verification" (CoVe)** into your prompt. This forces the model to double-check its own work before giving you the final answer.

The 3-Step Verification Prompt

1. **Extraction:** "First, identify the index of the very last row in the file."
2. **Validation:** "Cross-reference that index with the total number of entries found. If they do not match, re-scan the file in chunks of 500."
3. **Final Count:** "Only after verifying the last index, provide the final row count."

---

3. Dealing with Guardrails

If guardrails are causing the "1000 row" cutoff, it's often a **context window** issue. The platform might be hitting a "soft limit" where it stops processing to prevent a timeout.

How to "Bypass" the Truncation:

- **Chunking:** Ask the model to analyze the file in segments.
  - *Prompt:* "Analyze rows 1-900 first, then 901-1900. Summarize each chunk separately, then combine the totals."
- **Metadata Check:** Ask for the file metadata before the analysis.
  - *Prompt:* "Before analyzing the content, tell me the exact file size in KB and the total line count according to the file system."

---

4. Comparison of Methods

Method	Accuracy	Effort	Why it works

Pure LLM Prompt	Low (~60%)	Minimal	Model "hallucinates" a round number (like 1000).
Chain of Verification	Medium (85%)	Moderate	Forces the model to look at the beginning and end of the file.
Python/Code Execution	**100%**	High	Uses math/logic instead of text prediction.

Export to Sheets

Summary Feedback Mechanism

If you want to build this into a permanent "fix," use a **multi-agent approach** (if your platform allows):

1. **Agent A (Executor):** Counts the rows using a script.
2. **Agent B (Checker):** Takes the output of Agent A and manually checks the last 5 rows of the CSV to see if the index matches.
3. **Feedback:** If Agent B finds a mismatch, it triggers Agent A to re-run the process with a different encoding or delimiter.